Algorithms

Lecture 10-11

(Sorting Revisited and extended)

By: Sarfraz

Few slides taken from Sara/Nimra presentation

Quick Sort

```
QUICKSORT(A, p, r)
1   if p < r
2       q = PARTITION(A, p, r)
3       QUICKSORT(A, p, q − 1)
4       QUICKSORT(A, q + 1, r)
```

Quick Sort

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Worst/Best Case Partition???

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Worst/Best Case Partition???

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

Worst CASE PARTITION

$$T(n) = T(n-1) + T(0) + \Theta(n)$$

Best CASE PARTITION

$$T(n) = 2T(n/2) + \Theta(n)$$

Recursive Tree of QuickSort??? Even if it is skewed **IN PROPORTION**??? Will lead to???

What if the split happens with 1/10 on one side and 9/10 on other side of recursive tree???

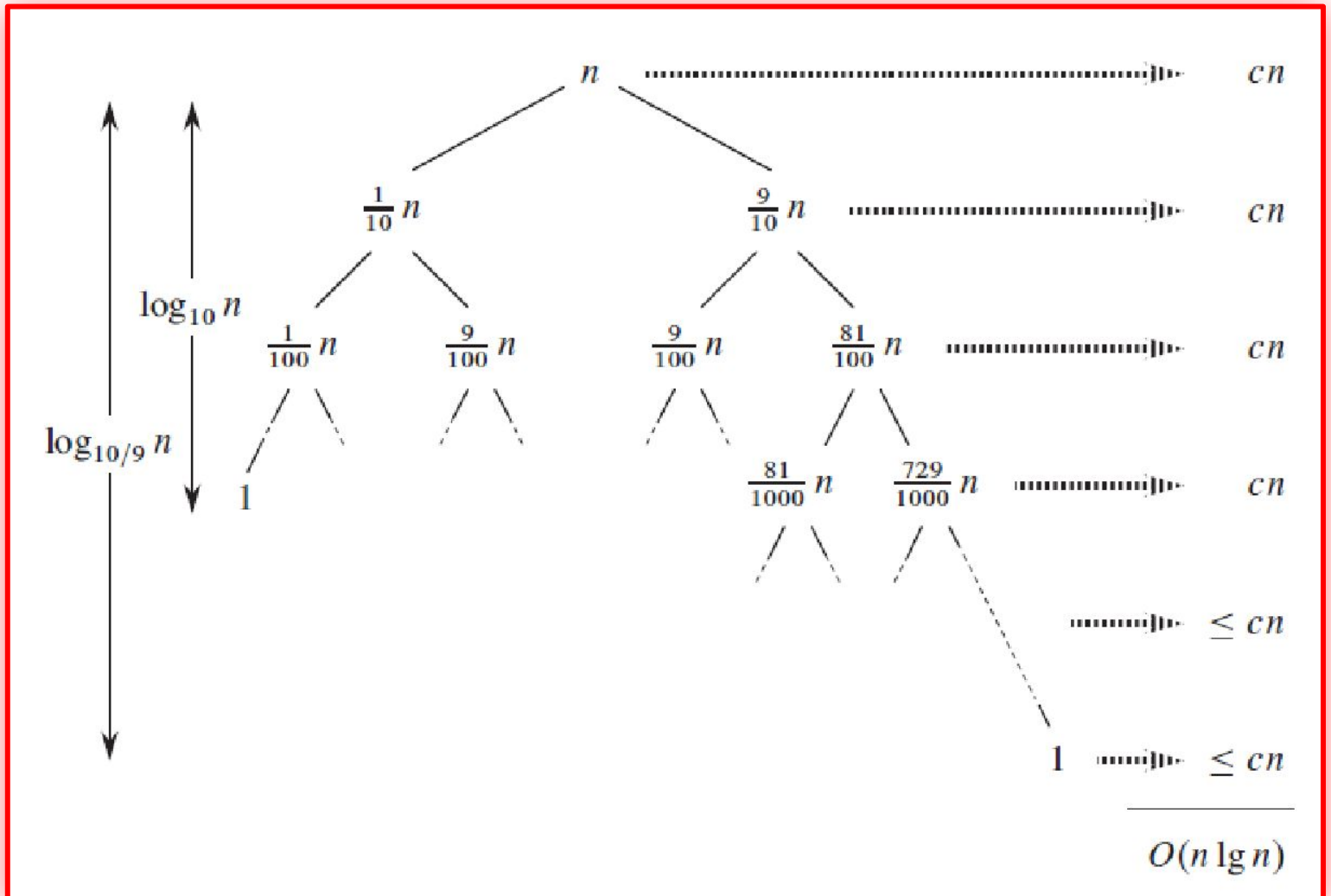
Recurrence???

Recursive Tree of QuickSort??? Even if it is skewed **IN PROPORTION**??? Will lead to???

What if the split happens with 1/10 on one side and 9/10 on other side of recursive tree???

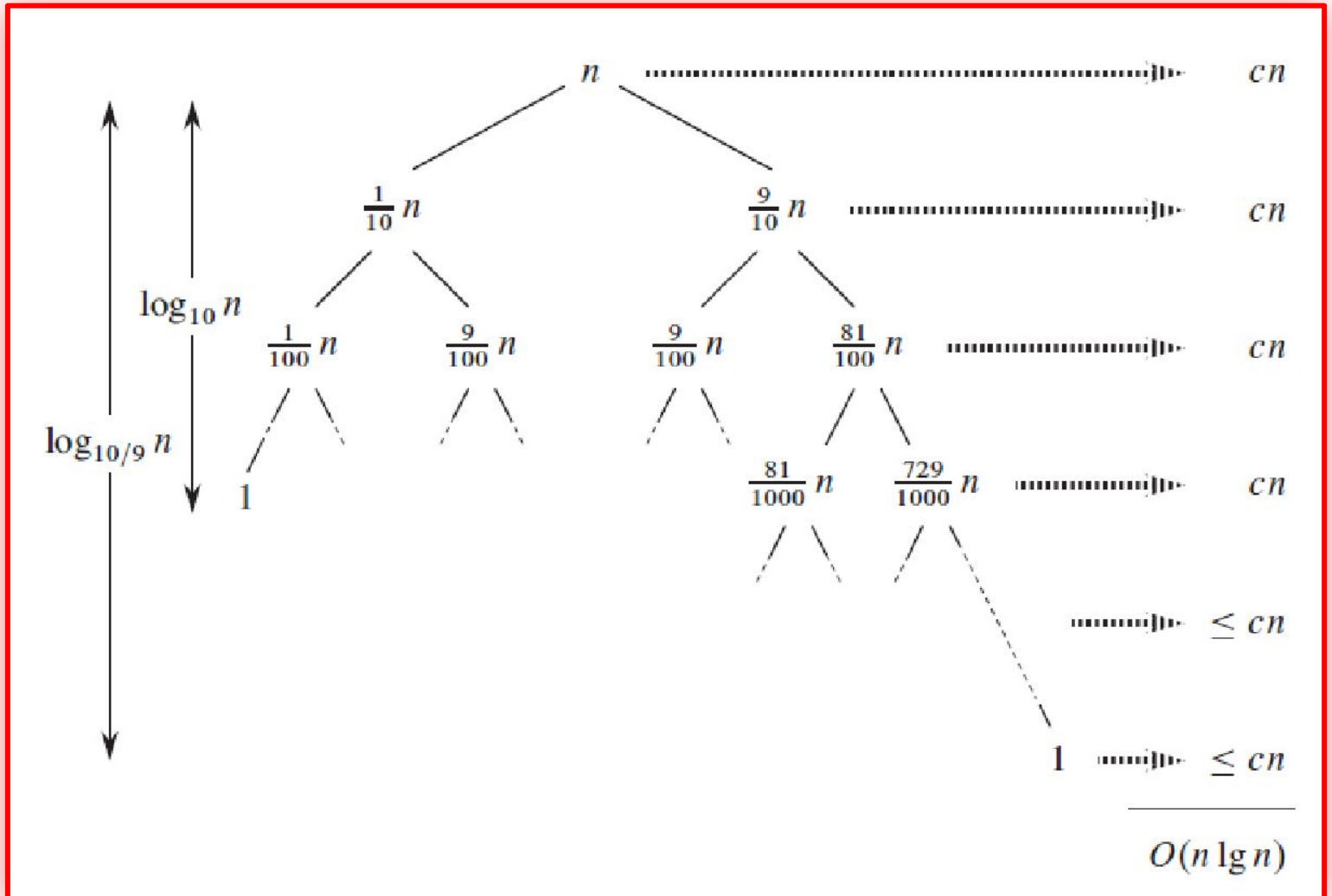
$$T(n) = T(9n/10) + T(n/10) + cn$$

Recursive Tree of QuickSort??? Even if it is skewed **IN PROPORTION**??? Will lead to???



$$T(n) = T(9n/10) + T(n/10) + cn$$

Recursive Tree of QuickSort??? Even if it is skewed **IN PROPORTION**??? Will lead to???



$$T(n) = T(9n/10) + T(n/10) + cn$$

Still **O(n log n)**

Randomized Quick-Sort

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4       if A[j] ≤ x
5            i = i + 1
6            exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2       q = RANDOMIZED-PARTITION(A, p, r)
3       RANDOMIZED-QUICKSORT(A, p, q − 1)
4       RANDOMIZED-QUICKSORT(A, q + 1, r)
```

Randomized Select

```
RANDOMIZED-SELECT(A, p, r, i)
1   if p == r
2       return A[p]
3   q = RANDOMIZED-PARTITION(A, p, r)
4   k = q − p + 1
5   if i == k              // the pivot value is the answer
6       return A[q]
7   elseif i < k
8       return RANDOMIZED-SELECT(A, p, q − 1, i)
9   else return RANDOMIZED-SELECT(A, q + 1, r, i − k)
```

CAN WE MAKE DETERMINISTIC QUICK-SORT???

- WITH ENSURETY THAT WE HAVE **O(N Log N) Complexity**???

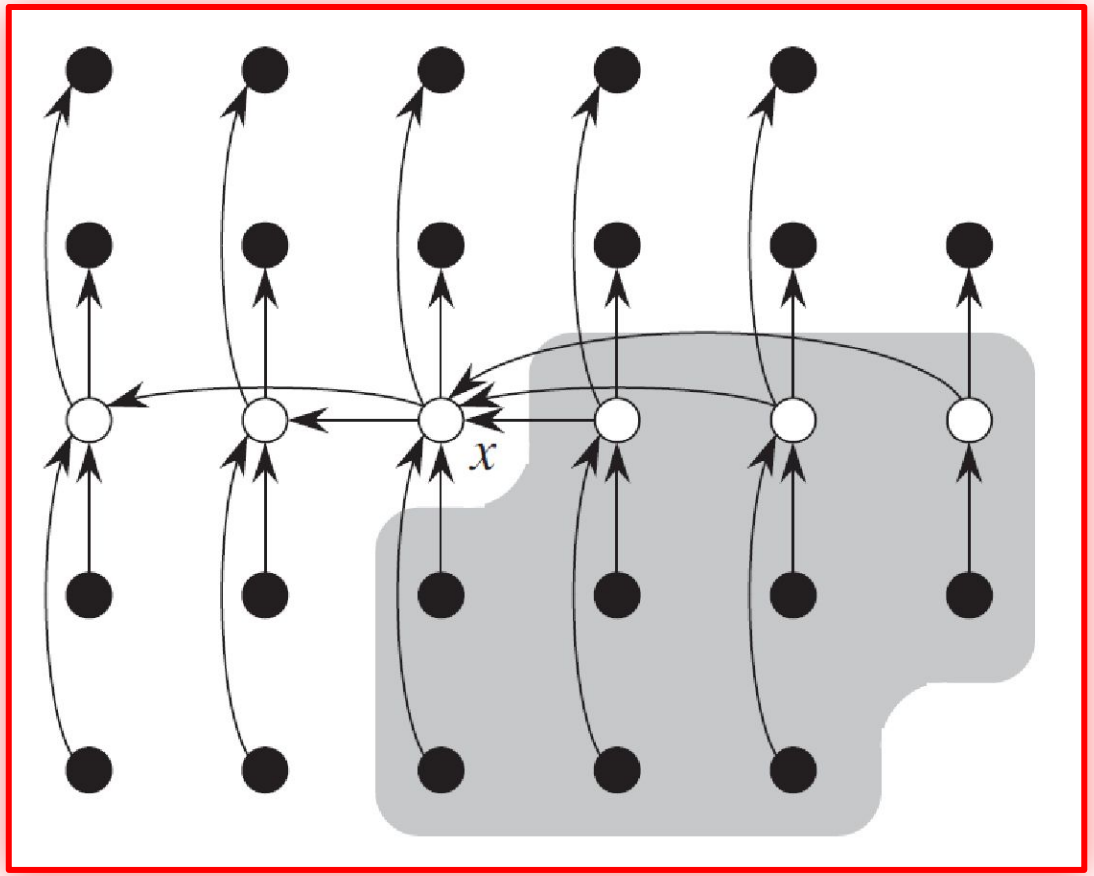
CAN WE MAKE DETERMINISTIC QUICK-SORT???

- WITH ENSURETY THAT WE HAVE **O(N Log N) Complexity**???
- WHAT Problem if we solve will ensure O(N Log N) Complexity???

K’th Smallest Element in Linear Time???

- A Generalized Version of Median Searching?

Median of Median??? In Linear Time???



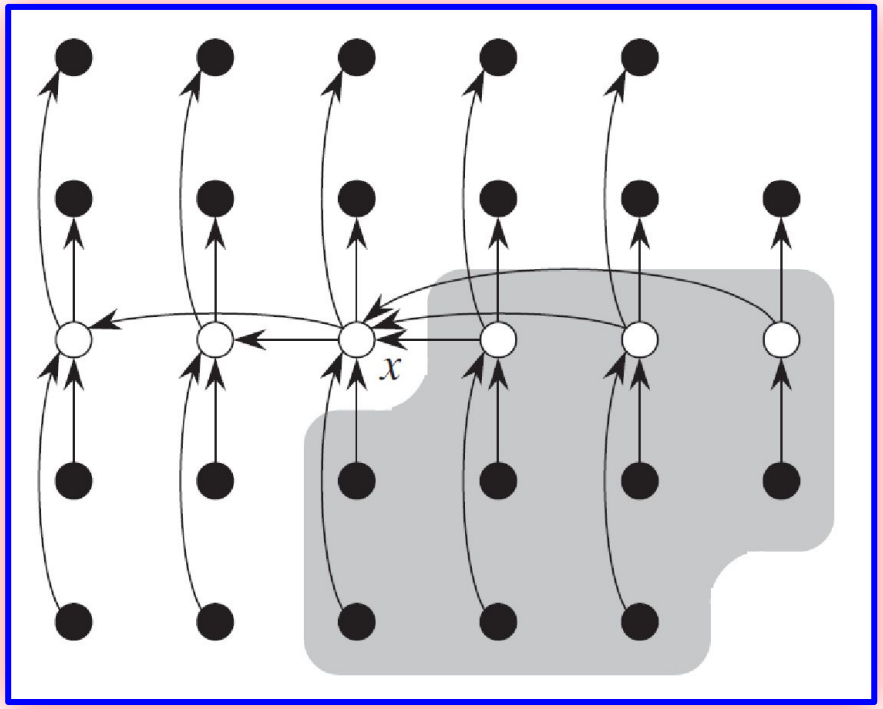
- Sort each n/5 problem
- Complexity for sorting each n/5 problem = 25(n/5) = 5N = O(N)

Median of Median??? In Linear Time???

Algorithm for Selection

A storm of medians

```
select(A, j):
    Form lists L_1, L_2, ..., L_⌈n/5⌉ where L_i = {A[5i − 4], ..., A[5i]}
    Find median b_i of each L_i using brute-force
    B = [b_1, b_2, ..., b_⌈n/5⌉]
    b = select(B, ⌈n/10⌉)
    Partition A into A_less and A_greater using b as pivot
    if (|A_less|) = j return b
    else if (|A_less|) > j)
        return select(A_less, j)
    else
        return select(A_greater, j − |A_less|)
```



- Sort each n/5 problem
- Complexity for sorting each n/5 problem = 25(n/5) = 5N = O(N)

Median of Median??? In Linear Time???

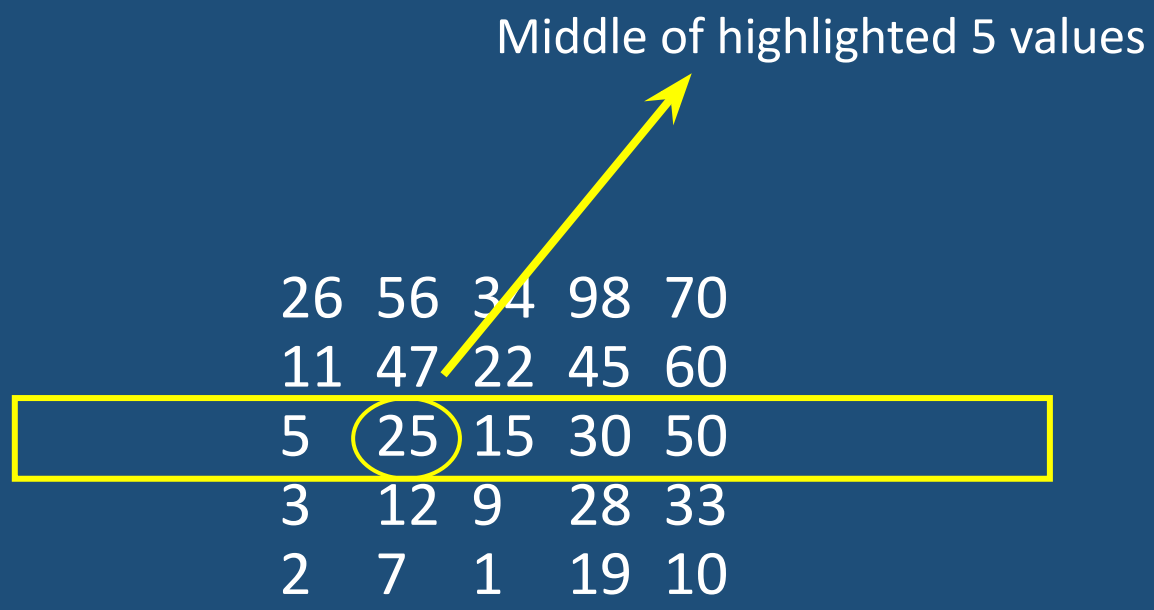
3	2	5	26	11	12	47	25	56	7	34	22	15	9	1	98	45	30	28	19	10	33	70	50	60

n = 25

- Sort each n/5 problem
- Complexity for sorting each n/5 problem = 25(n/5) = 5N = O(N)

26	56	34	98	70
11	47	22	45	60
5	25	15	30	50
3	12	9	28	33
2	7	1	19	10

Median of Median??? In Linear Time???

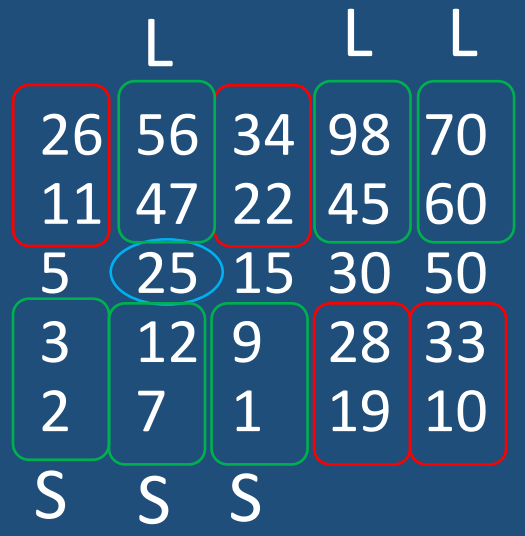


Median of Median??? In Linear Time???

We are sure about green regions

BUT

We are sure not sure about red region

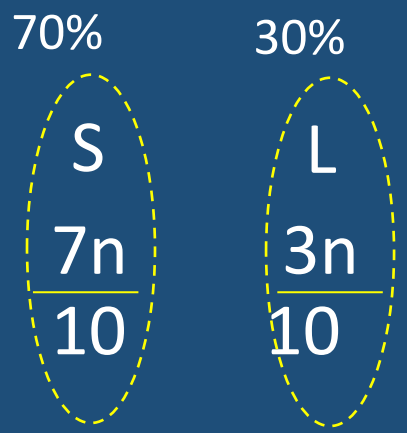


Median of Median??? In Linear Time???

For the sake of understanding let say

In worst case scenario we have smaller values from 25 in red region

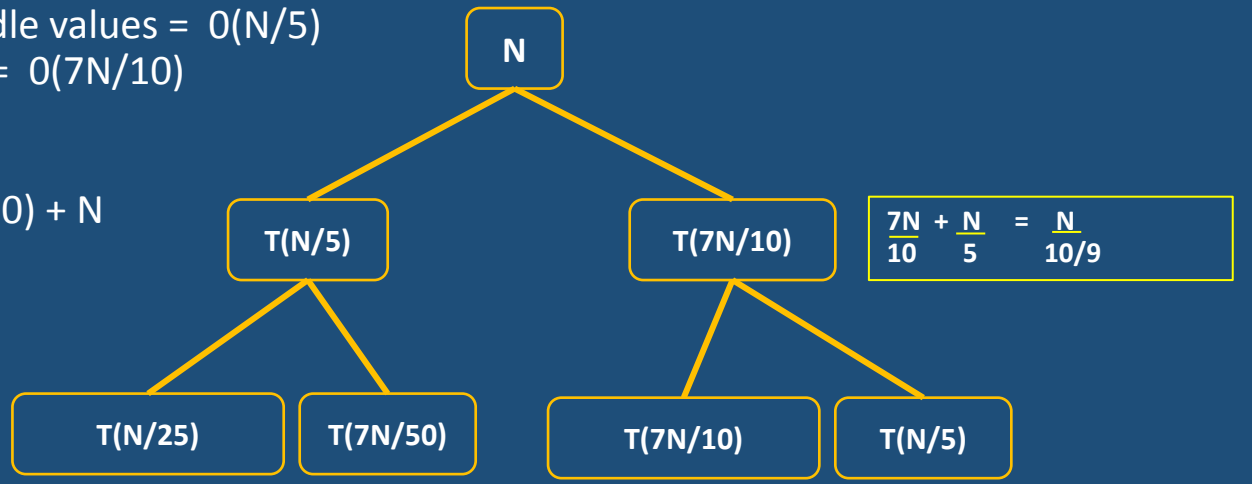
26	56	34	98	70
11	47	22	45	60
5	**25**	15	30	50
3	12	9	28	33
2	7	1	19	10



Median of Median??? In Linear Time???

Sorting chunks = O(N)
Finding median of middle values = 0(N/5)
Finding 5N/10 in 75% = 0(7N/10)

Recurrence Relation:
T(n) = T(N/5) + T(7N/10) + N



$\frac{7N}{10} + \frac{N}{5} = \frac{N}{10/9}$

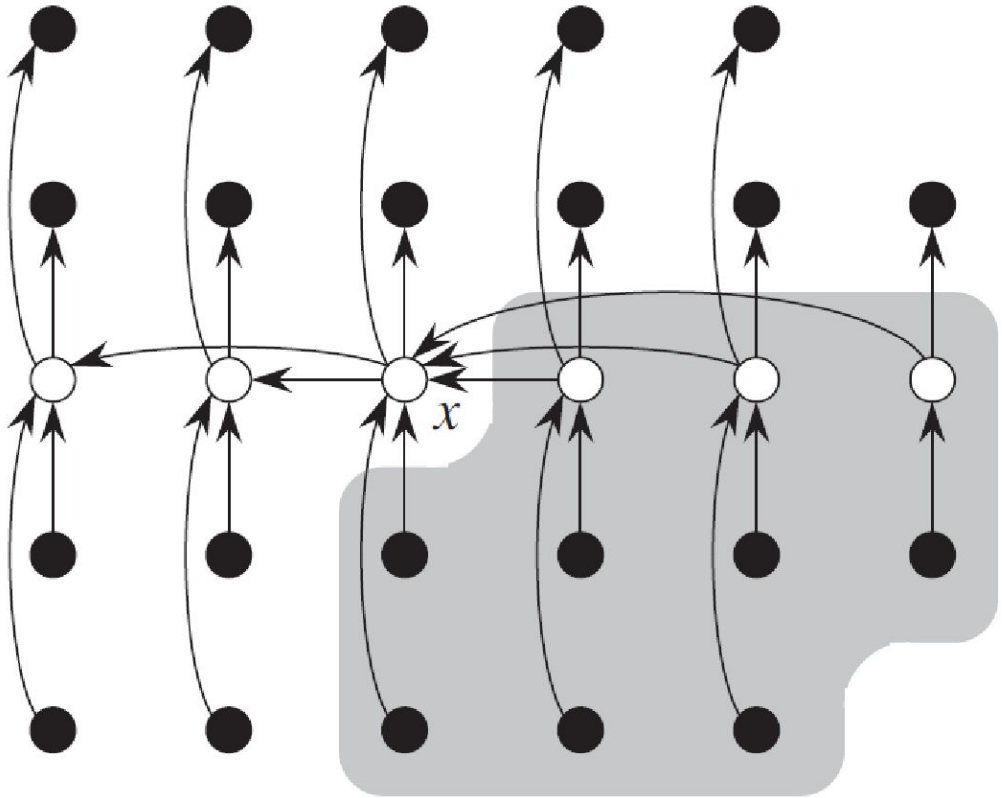
$\frac{N}{25} + \frac{7N}{50} + \frac{7N}{50} + \frac{49N}{100} = \frac{N}{100/81}$

$$N + \frac{N}{(10/9)^1} + \frac{N}{(10/9)^2} + \ldots\ldots\ldots\ldots + 1$$

This is decreasing geometric series so bounded by the first term

Complexity = O(N)

Median of Median??? In Linear Time???



$$T(n) \le \begin{cases} O(1) & \text{if } n < 140\,, \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{if } n \ge 140\,. \end{cases}$$

Median Search Problem?

Algorithm for Selection

A storm of medians

```
select(A, j):
    Form lists L_1, L_2, ..., L_⌈n/5⌉ where L_i = {A[5i − 4], ..., A[5i]}
    Find median b_i of each L_i using brute-force
    B = [b_1, b_2, ..., b_⌈n/5⌉]
    b = select(B, ⌈n/10⌉)
    Partition A into A_less and A_greater using b as pivot
    if (|A_less|) = j return b
    else if (|A_less|) > j)
        return select(A_less, j)
    else
        return select(A_greater, j − |A_less|)
```

https://onlinegdb.com/WNSeetQJ5

Can Sorting be done better then O(N log N)

- Given a range of 1-n numbers can you sort them in linear time?

Sorting in Linear Time

```
COUNTING-SORT(A, B, k)
 1  let C[0..k] be a new array
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to A.length
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  for j = A.length downto 1
11      B[C[A[j]]] = A[j]
12      C[A[j]] = C[A[j]] − 1
```

Sorting in Linear Time

```
COUNTING-SORT(A, B, k)
 1  let C[0..k] be a new array
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to A.length
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  for j = A.length downto 1
11      B[C[A[j]]] = A[j]
12      C[A[j]] = C[A[j]] − 1
```

COMPLEXITY???

Sorting in Linear Time

```
COUNTING-SORT(A, B, k)
 1  let C[0..k] be a new array
 2  for i = 0 to k
 3      C[i] = 0
 4  for j = 1 to A.length
 5      C[A[j]] = C[A[j]] + 1
 6  // C[i] now contains the number of elements equal to i.
 7  for i = 1 to k
 8      C[i] = C[i] + C[i − 1]
 9  // C[i] now contains the number of elements less than or equal to i.
10  for j = A.length downto 1
11      B[C[A[j]]] = A[j]
12      C[A[j]] = C[A[j]] − 1
```

Is It Stable Sort???

1. Describe an algorithm that, given n integers in the range 0 to k, preprocesses its input and then answers any query about how many of the n integers fall into a range $[a\,..\,b]$ in $O(1)$ time. Your algorithm should use $\Theta(n + k)$ preprocessing time.

1. Describe an algorithm that, given n integers in the range 0 to k, preprocesses its input and then answers any query about how many of the n integers fall into a range $[a\,..\,b]$ in $O(1)$ time. Your algorithm should use $\Theta(n + k)$ preprocessing time.

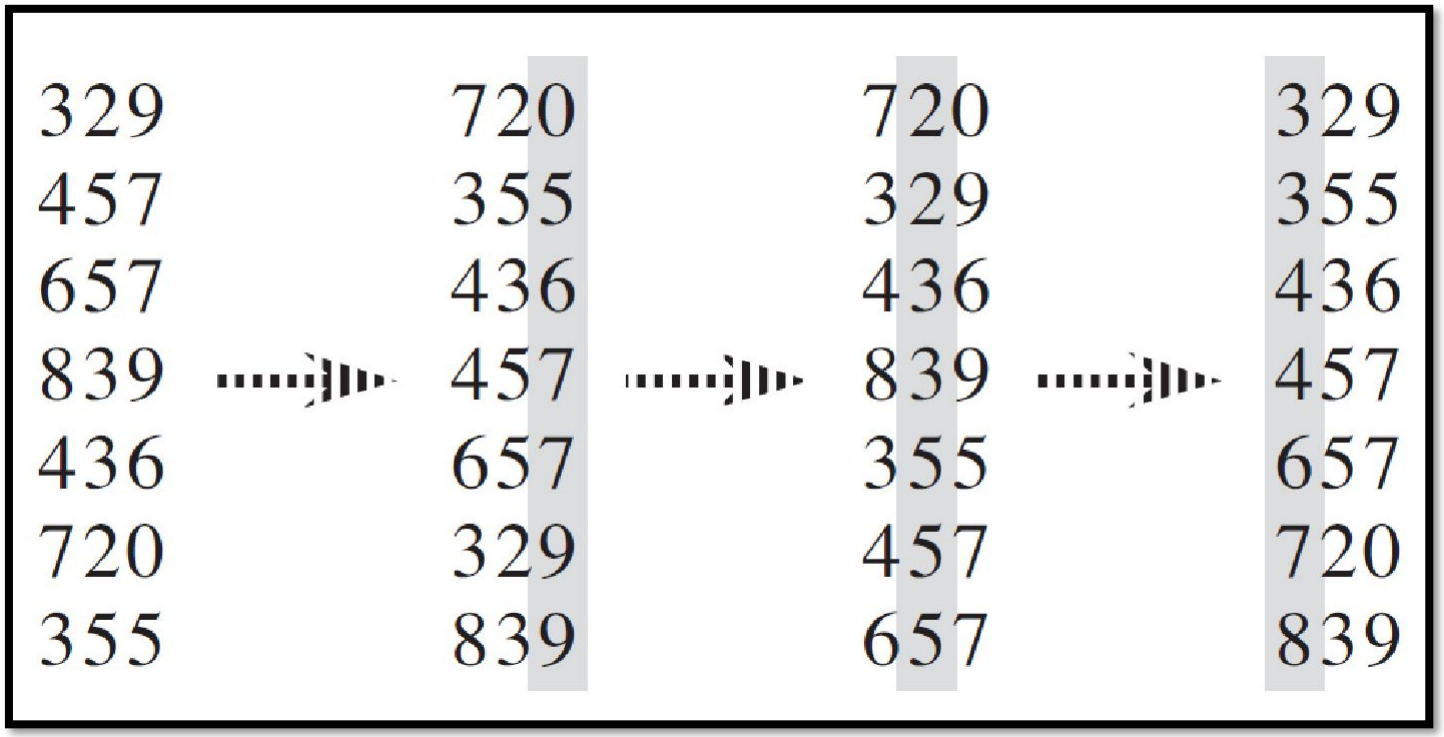
2. Sort them??? NOTE THAT VALUES are few but range could be large???

329
457
657
839
436
720
355

1. Describe an algorithm that, given n integers in the range 0 to k, preprocesses its input and then answers any query about how many of the n integers fall into a range $[a\,..\,b]$ in $O(1)$ time. Your algorithm should use $\Theta(n + k)$ preprocessing time.

2.

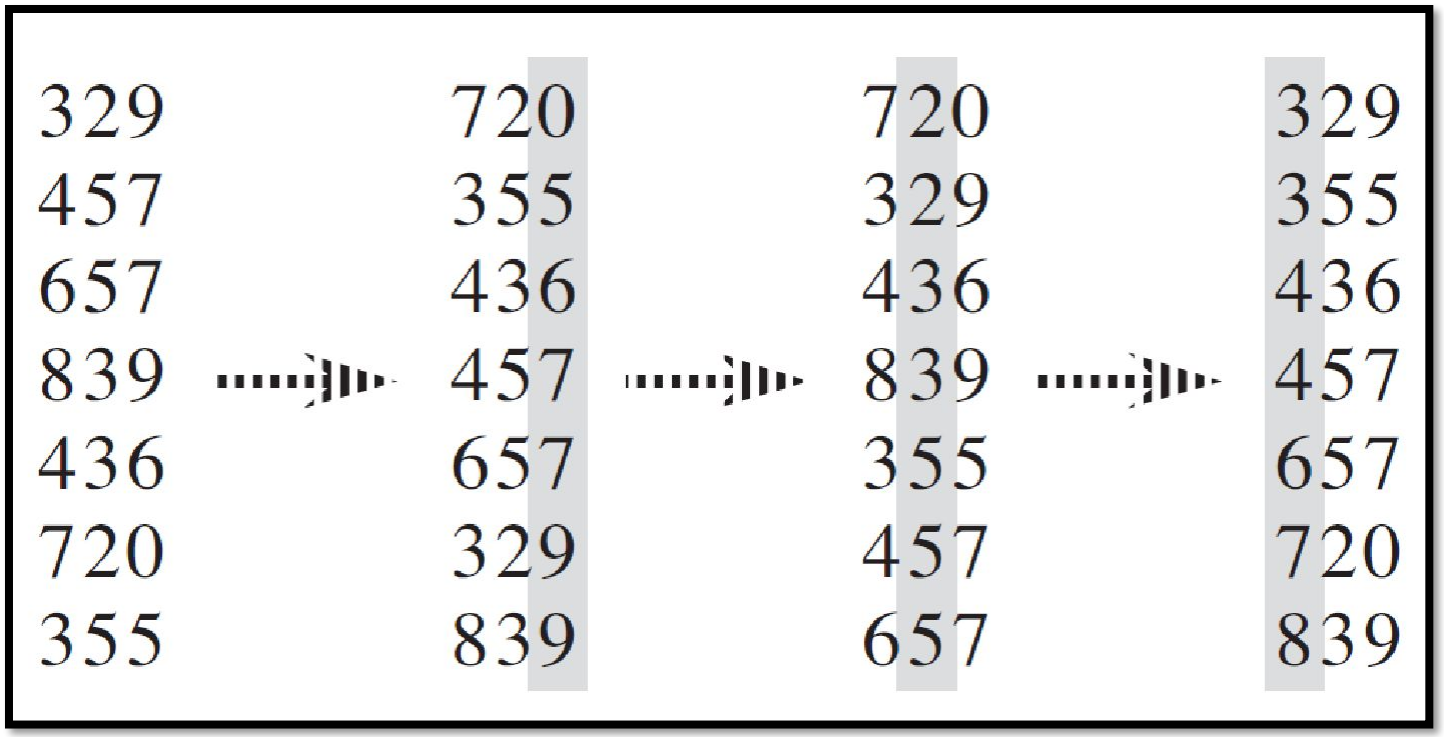
329
457
657
839
436
720
355



1. Describe an algorithm that, given n integers in the range 0 to k, preprocesses its input and then answers any query about how many of the n integers fall into a range $[a\,..\,b]$ in $O(1)$ time. Your algorithm should use $\Theta(n + k)$ preprocessing time.

2.

329
457
657
839
436
720
355



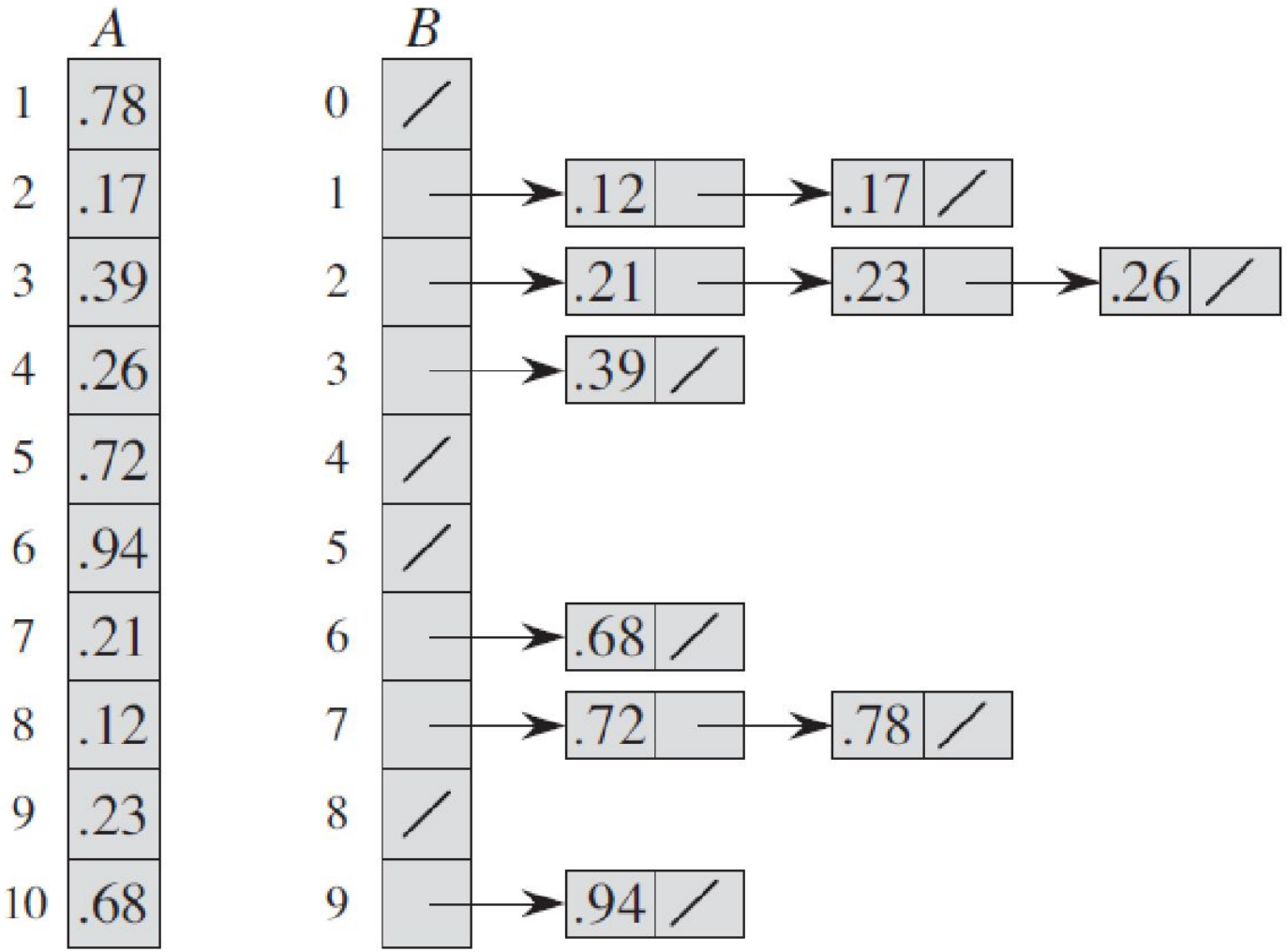
RADIX-SORT(A, d)

1 **for** $i = 1$ **to** d
2 use a stable sort to sort array A on digit i

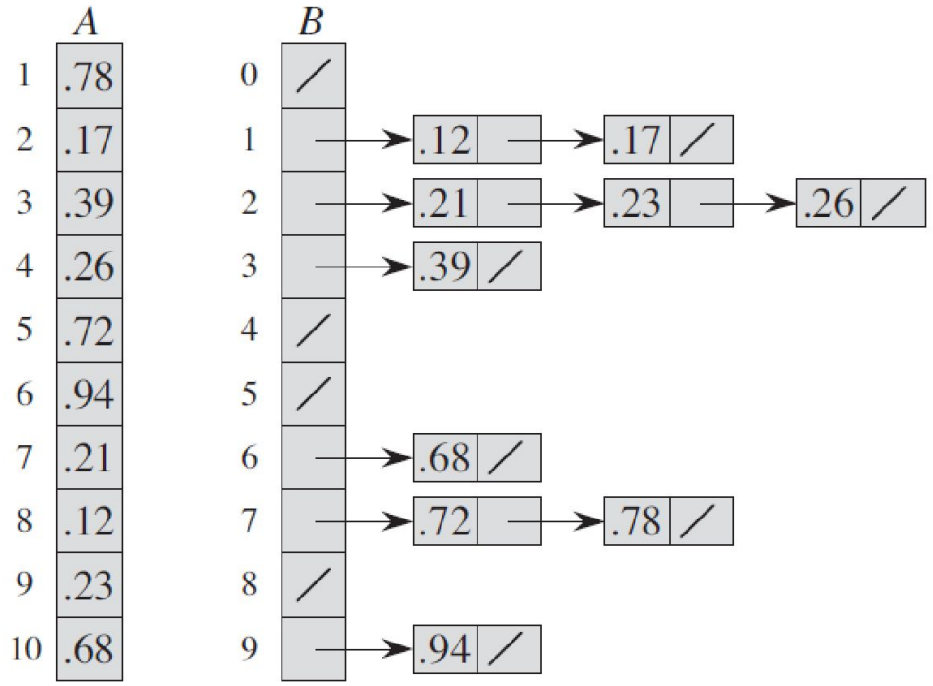
Problem

Show how to sort n integers in the range 0 to $n^3 - 1$ in $O(n)$ time.

Bucketing Idea???



Bucketing Idea???



BUCKET-SORT(A)

1 let $B[0..n-1]$ be a new array
2 $n = A.length$
3 **for** $i = 0$ **to** $n-1$
4 make $B[i]$ an empty list
5 **for** $i = 1$ **to** n
6 insert $A[i]$ into list $B[\lfloor nA[i] \rfloor]$
7 **for** $i = 0$ **to** $n-1$
8 sort list $B[i]$ with insertion sort
9 concatenate the lists $B[0], B[1], \ldots, B[n-1]$ together in order